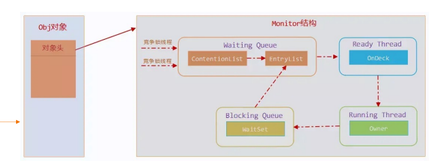
synchronized锁升级

> 初始化：创建java对象obj，mark word处于匿名偏向状态
> 获取偏向锁：线程1执行obj同步代码前，获取偏向锁（将线程id cas到obj mark word，并在线程栈关联一条lock record），持锁执行同步代码
　　线程1继续执行obj其他同步代码，发现锁已偏向自己，只需关联一条lock record，无需其他操作（提升性能）
　　线程1执行完obj所有同步代码，解锁（取消线程栈关联的lock records，obj mark word先不管）
> 升级轻量锁：线程2执行obj同步代码，发现锁偏向线程1，构造VM_RevokeBias提交到VMOperationQueue，然后阻塞等待jvm通知
　　> 如果升级前线程1已消亡 or 已执行完同步代码，升级后：线程2获取轻量锁成功，持锁执行同步代码，执行完解锁将obj mark word置为无锁状态
　　> 如果升级前线程1还在执行同步代码，升级后：a)线程1继续持锁执行，执行完解锁将obj mark word置为无锁状态　b)线程2获取锁失败，膨胀成重量锁
> 膨胀重量锁：线程2获取轻量锁失败时：1)把线程1的轻量锁替换为重量锁
　　2)抢锁：CAS -> 自旋 -> 进入monitor的_cxq阻塞等锁

> VMThread持续监测VMOperationQueue，发现有VM_Operation时取出执行
> 如果VM_Operation需要在安全点执行，等所有java线程运行到安全点时阻塞挂起，开始执行VM_RevokeBias
　检查线程1是否还在执行obj同步代码：如果是，修改线程1栈中第1条Lock Record，并将其地址更新到obj mark word中（这样改完，效果相当于线程1获取了obj轻量锁）
　　如果否，obj mark word修改成无锁状态（清除线程1解锁后遗留的mark word）
> 通知等待的java线程
> 为什么一定要全局停顿后再升级？因为此时不能确定biased thread是否还在执行同步代码，要在safe point时查看其线程栈才能确定，从而采取不同的操作

> 将obj mark word cas为INFLATING，防止多线程冲突（此时线程1也无法释放锁）
> 构造ObjectMonitor实例m，设置属性：_owner为obj的轻量锁持有者（即mark word中的lock record），_object为obj
> 将m地址更新到obj mark word
> 为什么不用全局停顿了？因为此时能确定线程1还在执行同步代码



```
public void test() { //场景1：synchronized修饰代码块
  synchronized (this) {
    System.out.println("This is just a test run.");
  }
}
```

编译后的byte code:

```
  public test()V
   L4
    LINENUMBER 5 L4
    ALOAD 0
    DUP
    ASTORE 1
    MONITORENTER
   L0
    GETSTATIC java/lang/System.out : Ljava/io/PrintStream;
    LDC "This is just a test run."
    INVOKEVIRTUAL java/io/PrintStream.println (Ljava/lang/String;)V
    ALOAD 1
    MONITOREXIT
   L1
     ...
```

```
public class SynchronizedTest { //场景2：synchronized修饰方法
    public synchronized static void test1() {} //类方法
    public synchronized  void test2() {} //实例方法
}
```

编译后的byte code:

```
  //access flags 0x29 被synchronized修饰的方法，access flag带有标记
  public static synchronized test1()V
   L0
    LINENUMBER 4 L0
    RETURN
    MAXSTACK = 0
    MAXLOCALS = 0

  //access flags 0x21 被synchronized修饰的方法，access flag带有标记
  public synchronized test2()V
   L0
    LINENUMBER 5 L0
    RETURN
   L1
    LOCALVARIABLE this LThreadTest/SynchronizedTest; L0 L1 0
    MAXSTACK = 0
    MAXLOCALS = 1
```

```
src/hotspot/share/interpreter/zero/bytecodeInterpreter.cpp::run()
//jvm执行方法调用前，检查方法的access flag，如果有synchronized标记，先获取对象锁，再调用
-> switch(istate->msg()) {
     CASE(method_entry): {
       if (METHOD->is_synchronized()) { //1.获取对象锁
         oop rcvr; //获取锁的对象
         if (METHOD->is_static()) {
           rcvr = METHOD->constants()->pool_holder()->java_mirror(); //类的镜像
         } else {
           rcvr = LOCALS_OBJECT(0); //方法所属对象
         }
         BasicObjectLock* mon = &istate->monitor_base()[-1];
         mon->set_obj(rcvr);
         bool success = false;
         markOop mark = rcvr->mark();
         if (mark->has_bias_pattern()) { /* 尝试获取偏向锁*/ }
         if (!success) { /*尝试获取轻量级锁偏向锁 }
       }
     }
   }
   switch (opcode) {
     CASE(_invokevirtual):
     CASE(_invokespecial):
     CASE(_invokestatic): { ... } //2.调用方法
   }
```

```
src/hotspot/share/interpreter/zero/bytecodeInterpreter.cpp::run(interpreterState istate) //解释器执行bytecode
-> switch(opcode) { //读取opcode，执行相关操作
  CASE(_monitorenter): { //获取锁
    oop lockee = STACK_OBJECT(-1); //锁对象，即synchronized($锁对象$)中的锁对象
    CHECK_NULL(lockee);
    //1. 在当前线程栈中找到第1个空闲的Lock Record
    BasicObjectLock* limit = istate->monitor_base();
    BasicObjectLock* most_recent = (BasicObjectLock*) istate->stack_base();
    BasicObjectLock* entry = NULL;
    while (most_recent != limit ) {
      if (most_recent->obj() == NULL) entry = most_recent;
      else if (most_recent->obj() == lockee) break;
      most_recent++;
    }
    //2. 找到了
    if (entry != NULL) {
      entry->set_obj(lockee); //锁对象保存到Lock Record
      int success = false; //偏向锁成功标志位
      markOop mark = lockee->mark();
      intptr_t hash = (intptr_t) markOopDesc::no_hash;
      uintptr_t epoch_mask_in_place = (uintptr_t)markOopDesc::epoch_mask_in_place;

      //2.1 implies UseBiasedLocking, 偏向模式（锁对象头mark word的最后3位bit是101）
      if (mark->has_bias_pattern()) {
        ...
      }
      //2.2 traditional lightweight locking, 轻量级锁
      if (!success) {
        markOop displaced = lockee->mark()->set_unlocked(); //构造一个无锁状态的mark word，后续解锁用
        entry->lock()->set_displaced_header(displaced); //保存到Lock Record中
        bool call_vm = UseHeavyMonitors;
        if (call_vm || Atomic::cmpxchg_ptr(entry, lockee->mark_addr(), displaced) != displaced) {
          //失败，锁对象不是无锁状态
          if (!call_vm && THREAD->is_lock_owned((address) displaced->clear_lock_bits())) {
            //锁重入了，将Displaced Mark Word置空
            entry->lock()->set_displaced_header(NULL);
          } else {
            CALL_VM(InterpreterRuntime::monitorenter(THREAD, entry), handle_exception);
          }
        }
      }
      UPDATE_PC_AND_TOS_AND_CONTINUE(1, -1);
    } else {
      //3. 没找到，重新执行
      istate->set_msg(more_monitors);
      UPDATE_PC_AND_RETURN(0); // Re-execute
    }
  }


  CASE(_monitorexit): { //释放锁
    oop lockee = STACK_OBJECT(-1);
    BasicObjectLock* limit = istate->monitor_base();
    BasicObjectLock* most_recent = (BasicObjectLock*) istate->stack_base();
    //遍历栈的Lock Record
    while (most_recent != limit ) {
      if ((most_recent)->obj() == lockee) { //Lock Record关联的是当前lockee
        BasicLock* lock = most_recent->lock();
        markOop header = lock->displaced_header(); //1. 获取lockee被锁前的mark word
        most_recent->set_obj(NULL); //2. 释放Lock Record
        //偏向模式仅释放Lock Record就好了，其他模式
        if (!lockee->mark()->has_bias_pattern()) {
          bool call_vm = UseHeavyMonitors;
          if (header != NULL || call_vm) {
            //3. 尝试恢复lockee的mark word
            if (call_vm || Atomic::cmpxchg_ptr(header, lockee->mark_addr(), lock) != lock) {
              //4. CAS失败，或重量锁会走到这里，先将obj还原
              most_recent->set_obj(lockee);
              CALL_VM(InterpreterRuntime::monitorexit(THREAD, most_recent), handle_exception);
            }
          }
        }
        UPDATE_PC_AND_TOS_AND_CONTINUE(1, -1); //执行下一条命令
      }
      most_recent++; //5. 处理下一条Lock Record
    }
    CALL_VM(InterpreterRuntime::throw_illegal_monitor_state_exception(THREAD), handle_exception);
  }
```

```
void InterpreterRuntime::monitorexit(JavaThread* thread, BasicObjectLock* elem) {
  Handle h_obj(thread, elem->obj());
  if (elem == NULL || h_obj()->is_unlocked()) THROW(vmSymbols::java_lang_IllegalMonitorStateException());
  ObjectSynchronizer::slow_exit(h_obj(), elem->lock(), thread);
  elem->set_obj(NULL);
}
```

```
void ObjectSynchronizer::slow_exit(oop object, BasicLock* lock, TRAPS)
-> fast_exit (oop object, BasicLock* lock, TRAPS) {
  markOop dhw = lock->displaced_header();
  markOop mark = object->mark();
  //1. 如果是轻量锁解锁：恢复lockee的mark word
  if (mark == (markOop) lock) {
    if ((markOop) Atomic::cmpxchg_ptr (dhw, object->mark_addr(), mark) == mark) return;
  }
  //2. 是重量级锁，或轻量级锁解锁时出现竞争
  ObjectSynchronizer::inflate(THREAD, object, inflate_cause_vm_internal)->exit(true, THREAD);
}
```

```
void ATTR ObjectMonitor::exit(bool not_suspended, TRAPS) {
  Thread * Self = THREAD ;
  for (;;) {
    if (Knob_ExitPolicy == 0) {
      //1. Drop the lock (将Monitor::_owner属性置空，如果某个线程正在自旋抢占该锁，会抢占成功)
      OrderAccess::release_store_ptr (&_owner, NULL) ;
      OrderAccess::storeload() ;  //持锁期间的所有数据修改，修改立即生效
      //2. 如果没有等待获取锁的线程，直接返回
      if (((intptr_t(_EntryList)|intptr_t(_cxq)) == 0 || _succ != NULL) return;
      //3. 其他线程已经抢到了锁，直接返回
      if (Atomic::cmpxchg_ptr (THREAD, &_owner, NULL) != NULL) return;
    } else {
      if ((intptr_t(_EntryList)|intptr_t(_cxq)) == 0 || _succ != NULL) {
        OrderAccess::release_store_ptr (&_owner, NULL);
        OrderAccess::storeload();
        if (_cxq == NULL || _succ != NULL) return;
        if (Atomic::cmpxchg_ptr (THREAD, &_owner, NULL) != NULL) return;
      }
    }

    ObjectWaiter * w = NULL;
    int QMode = Knob_QMode; //唤醒策略，默认值是0
    if (QMode == 2 && _cxq != NULL) { w = _cxq; ExitEpilog (Self, w); return; } //唤醒_cxq的线程，并返回
    if (QMode == 3 && _cxq != NULL) { ... } //_cxq的元素加到_EntryList末尾
    if (QMode == 4 && _cxq != NULL) { ... } //_cxq的元素加到_EntryList头部

    w = _EntryList;
    if (w != NULL) {
      ExitEpilog (Self, w); //唤醒_EntryList中的第1个线程
      return;
    }
    ...
  }
}
```

> displaced是lockee无锁时的mark word
> Atomic::cmpxchg_ptr()原子性地执行了以下逻辑
　> 如果lockee mark word是无锁状态，将其替换成Lock Record
　> 否则，返回lockee mark word

```
if (mark->has_bias_pattern()) {
  uintptr_t thread_ident = (uintptr_t)istate->thread(); //当前线程id
  uintptr_t anticipated_bias_locking_value = (((uintptr_t)lockee->klass()->prototype_header() | thread_ident) ^ (uintptr_t)mark) & ~((uintptr_t) markOopDesc::age_mask_in_place);
  //2.1.1. 锁偏向自己，直接返回 already biased towards this thread, nothing to do
  if (anticipated_bias_locking_value == 0) success = true;
  else if ((anticipated_bias_locking_value & markOopDesc::biased_lock_mask_in_place) != 0) {
    //2.1.2. class的偏向模式关闭，尝试撤销偏向 try revoke bias
    markOop header = lockee->klass()->prototype_header();
    if (hash != markOopDesc::no_hash) header = header->copy_set_hash(hash);
    Atomic::cmpxchg_ptr(header, lockee->mark_addr(), mark); //将lockee的mark word恢复为class中的mark word，实现撤销
  }
  else if ((anticipated_bias_locking_value & epoch_mask_in_place) !=0) {
    //2.1.3. epoch不等于class中的epoch，时间戳过期，尝试重偏向 Try rebias
    markOop new_header = (markOop) ( (intptr_t) lockee->klass()->prototype_header() | thread_ident); //构造一个偏向当前线程的mark word
    if (hash != markOopDesc::no_hash) new_header = new_header->copy_set_hash(hash);
    if (Atomic::cmpxchg_ptr((void*)new_header, lockee->mark_addr(), mark) == mark) {} //替换lockee原来的mark word
    else { CALL_VM(InterpreterRuntime::monitorenter(THREAD, entry), handle_exception); } //失败，存在多线程竞争，调用monitorenter方法升级锁
    success = true;
  }
  else { //偏向别的线程 or 匿名偏向
    //2.1.4. try to bias towards thread in case object is anonymously biased
    markOop header = (markOop) ((uintptr_t) mark & ((uintptr_t)markOopDesc::biased_lock_mask_in_place | (uintptr_t)markOopDesc::age_mask_in_place | epoch_mask_in_place));
    if (hash != markOopDesc::no_hash) header = header->copy_set_hash(hash);
    markOop new_header = (markOop) ((uintptr_t) header | thread_ident); //构造一个偏向当前线程的mark word
    if (Atomic::cmpxchg_ptr((void*)new_header, lockee->mark_addr(), header) == header) {} //如果lockee当前是匿名偏向，偏向自己
    else { CALL_VM(InterpreterRuntime::monitorenter(THREAD, entry), handle_exception); } //失败，存在多线程竞争，调用monitorenter方法升级锁
     success = true;
  }
}
```

```
void InterpreterRuntime::monitorenter(JavaThread* thread, BasicObjectLock* elem) {
  Handle h_obj(thread, elem->obj());
  //1. 偏向锁出现多线程竞争
  if (UseBiasedLocking) ObjectSynchronizer::fast_enter(h_obj, elem->lock(), true, CHECK);
  //2. 轻量锁出现多线程竞争
  else ObjectSynchronizer::slow_enter(h_obj, elem->lock(), CHECK);
}
```

```
void ObjectSynchronizer::slow_enter(Handle obj, BasicLock* lock, , TRAPS) {
  markOop mark = obj->mark();
  //1. 无锁状态，直接加锁，并返回
  if (mark->is_neutral()) {
    lock->set_displaced_header(mark);
    if (mark == (markOop) Atomic::cmpxchg_ptr(lock, obj()->mark_addr(), mark)) return;
  } else if (mark->has_locker() && THREAD->is_lock_owned((address)mark->locker())) {
    //2. 有锁，但是重入，将displaced mark word置空，并返回
    lock->set_displaced_header(NULL);
    return;
  }
  //3. 存在多线程竞争，膨胀为重量级锁：为什么不用在安全点执行了？，然后尝试获取重量级锁
  lock->set_displaced_header(markOopDesc::unused_mark()); //将displace mark word设置为一个特殊值
  ObjectSynchronizer::inflate(THREAD, obj(), inflate_cause_monitor_enter)->enter(THREAD);

}
```

```
ObjectMonitor * ATTR ObjectSynchronizer::inflate(Thread * Self, oop object, const InflateCause cause){
  for (;;) { //自旋
    const markOop mark = object->mark();
    //1. 已经是重量级锁了，直接返回
    if (mark->has_monitor()) {
      ObjectMonitor * inf = mark->monitor();
      return inf;
    }
    //2. 膨胀中，继续等待 Some other thread is converting from stack-locked to inflated
    if (mark == markOopDesc::INFLATING()) {
      TEVENT (Inflate: spin while INFLATING);
      ReadStableMark(object);
      continue;
    }
    //3. 轻量级锁状态，尝试膨胀
    if (mark->has_locker()) {
      ObjectMonitor * m = omAlloc (Self); //获取一个可用的ObjectMonitor
      m->Recycle();
      m->_Responsible  = NULL;
      m->OwnerIsThread = 0;
      m->_recursions   = 0;
      m->_SpinDuration = ObjectMonitor::Knob_SpinLimit;
      //把锁对象的Mark Word设置为INFLATING
      markOop cmp = (markOop) Atomic::cmpxchg_ptr(markOopDesc::INFLATING(), object->mark_addr(), mark);
      if (cmp != mark) { //如果CAS失败，说明冲突了，继续等待
        omRelease (Self, m, true);
        continue;
      }
      //如果成功，设置monitor各字段
      markOop dmw = mark->displaced_mark_helper();
      m->set_header(dmw); //displaced mark word
      m->set_owner(mark->locker()); //Lock Record
      m->set_object(object);
      object->release_set_mark(markOopDesc::encode(m)); //将锁对象的mark word设置为重量级锁模式
      return m;
    }
    //4. 无锁，重置monitor，返回
    ObjectMonitor * m = omAlloc (Self);
    m->Recycle();
    m->set_header(mark);
    m->set_owner(NULL);
    m->set_object(object);
    m->OwnerIsThread = 1;
    m->_recursions   = 0;
    m->_Responsible  = NULL;
    m->_SpinDuration = ObjectMonitor::Knob_SpinLimit;
    if (Atomic::cmpxchg_ptr(markOopDesc::encode(m), object->mark_addr(), mark) != mark) { //替换mark word
      //失败，其他线程正在inflate，释放monitor锁
      m->set_object (NULL);
      m->set_owner  (NULL);
      m->OwnerIsThread = 0;
      m->Recycle();
      omRelease (Self, m, true);
      m = NULL;
      continue;
    }
    return m;
  }
}
```

```
void ObjectSynchronizer::fast_enter(Handle obj, BasicLock* lock, bool attempt_rebias) {
  if (UseBiasedLocking) {
    //1. 获取偏向锁失败（已偏向其他线程，偏向模式关闭，获取偏向锁时出现并发冲突）时，都会走到这里
    if (!SafepointSynchronize::is_at_safepoint()) {
      //正常的java线程，撤销或升级偏向锁
      BiasedLocking::Condition cond = BiasedLocking::revoke_and_rebias(obj, attempt_rebias, THREAD);
      if (cond == BiasedLocking::BIAS_REVOKED_AND_REBIASED) return; //2. 已获得偏向锁，返回
    } else {
      //vm线程
      BiasedLocking::revoke_at_safepoint(obj);
    }
  }
  //3. 偏向锁升级为轻量锁，开始抢占锁
  slow_enter (obj, lock, THREAD);
}
```

```
void ATTR ObjectMonitor::enter(TRAPS) {
  Thread * const Self = THREAD;
  void * cur;
  //1. 将monitor的_owner设置为自己，并返回
  cur = Atomic::cmpxchg_ptr (Self, &_owner, NULL);
  if (cur == NULL) return; //加锁成功，直接返回
  if (cur == Self) { _recursions++; return; } //重入锁
  if (Self->is_lock_owned ((address)cur)) { //当前线程轻量级锁膨胀为重量级锁
    _recursions = 1;
    _owner = Self;
    OwnerIsThread = 1;
    return;
  }
  //2. monitor被其他线程持有，需要抢占
  //2.1 先尝试自旋抢占，若成功，直接返回
  if (Knob_SpinEarly && TrySpin (Self) > 0) {
    Self->_Stalled = 0;
    return;
  }

  JavaThread * jt = (JavaThread *) Self;
  {
    //2.2 不断尝试获取锁，直到成功
    for (;;) {
      EnterI (THREAD);
      if (!ExitSuspendEquivalent(jt)) break; //获取锁后，跳出循环，返回
      exit (false, Self);
    }
  }
}
```

```
ObjectMonitor * ATTR ObjectSynchronizer::omAlloc (Thread * Self){
  for (;;) {
    ObjectMonitor * m;
    // 1: try to allocate from the thread's local omFreeList.
    m = Self->omFreeList;
    if (m != NULL) {
      Self->omFreeList = m->FreeNext;
      Self->omFreeCount --;
      if (MonitorInUseLists) {
        m->FreeNext = Self->omInUseList;
        Self->omInUseList = m;
        Self->omInUseCount ++;
      }else { m->FreeNext = NULL; }
      return m;
    }
    // 2: try to allocate from the global gFreeList
    // 3: allocate a block of new ObjectMonitors
  }
}
```

```
int ObjectMonitor::TrySpin_VaryDuration (Thread * Self) {
  //1. 固定次数自旋
  int ctr = Knob_FixedSpin; //Knob_FixedSpin固定自旋次数，JDK6之前默认是10，之后默认是0
  if (ctr != 0) {
    while (--ctr >= 0) {
      if (TryLock (Self) > 0) return 1; //抢占成功，返回1
      SpinPause ();
    }
    return 0; //自旋指定次数后，还是失败，返回0
  }
  //2. 自适应自旋
  for (ctr = Knob_PreSpin + 1; --ctr >= 0; ) { //Knob_PreSpin是10
    //2.1 抢占锁成功时，increase _SpinDuration，然后返回
    if (TryLock(Self) > 0) {
      int x = _SpinDuration;
      if (x < Knob_SpinLimit) { //默认值是5000
        if (x < Knob_Poverty) x = Knob_Poverty;
        _SpinDuration = x + Knob_BonusB;
      }
      return 1;
    }
    //2.2 否则，休眠一段时间
    SpinPause ();
  }
  ...
}
```

```
void ATTR ObjectMonitor::enterI (TRAPS) {
  Thread * Self = THREAD;
  if (TryLock (Self) > 0) return; //1. 试一次获取锁
  if (TrySpin (Self) > 0) return; //2. 试一次自旋
  ObjectWaiter node(Self);
  Self->_ParkEvent->reset();
  ObjectWaiter * nxt;
  for (;;) { //3. 当前线程加入_cxq
    node._next = nxt = _cxq;
    if (Atomic::cmpxchg_ptr (&node, &_cxq, nxt) == nxt) break;
    if (TryLock (Self) > 0) return;
  }
  if ((SyncFlags & 16) == 0 && nxt == NULL && _EntryList == NULL) {
    Atomic::cmpxchg_ptr (Self, &_Responsible, NULL);
  }
  //4. 不停地循环尝试获取锁
  for (;;) {
    if (TryLock (Self) > 0) break;
    if ((SyncFlags & 2) && _Responsible == NULL) {
      Atomic::cmpxchg_ptr (Self, &_Responsible, NULL);
    }
    //4.1. 当前线程先休眠一段时间
    if (_Responsible == Self || (SyncFlags & 1)) {
      Self->_ParkEvent->park ((jlong) RecheckInterval);
      RecheckInterval *= 8;
      if (RecheckInterval > 1000) RecheckInterval = 1000;
    } else {
      Self->_ParkEvent->park();
    }
    //4.2. 当前线程被唤醒，有人释放了锁，赶紧再试一次
    if (TryLock(Self) > 0) break;
    if ((Knob_SpinAfterFutile & 1) && TrySpin (Self) > 0) break;
  }
  //5. 循环结束，已经持有锁，重置一些变量，并返回
  UnlinkAfterAcquire (Self, &node);
  if (_succ == Self) _succ = NULL;
  if (_Responsible == Self) {
    _Responsible = NULL;
    OrderAccess::fence(); //
  }
  return;
}
```

```
void os::PlatformEvent::park() {
  int v ;
  for (;;) {
    v = _Event ;
    if (Atomic::cmpxchg (v-1, &_Event, v) == v) break ;
  }
  if (v == 0) {
    int status = pthread_mutex_lock(_mutex);
    ++ _nParked ;
    while (_Event < 0) {
      status = pthread_cond_wait(_cond, _mutex);
      if (status == ETIME) { status = EINTR; }
    }
    -- _nParked ;
    _Event = 0 ;
    status = pthread_mutex_unlock(_mutex);
    OrderAccess::fence();
  }
}
```

```
BiasedLocking::Condition BiasedLocking::revoke_and_rebias(Handle obj, bool attempt_rebias) {
  markOop mark = obj->mark();
  if (mark->is_biased_anonymously() && !attempt_rebias) {  }
  else if (mark->has_bias_pattern()) {
    Klass* k = obj->klass();
    markOop prototype_header = k->prototype_header();
    if (!prototype_header->has_bias_pattern()) {
      //class偏向模式关闭
      markOop biased_value = mark;
      markOop res_mark = (markOop) Atomic::cmpxchg_ptr(prototype_header, obj->mark_addr(), mark);
      return BIAS_REVOKED;
    } else if (prototype_header->bias_epoch() != mark->bias_epoch()) {
      //epoch过期
      if (attempt_rebias) {
        markOop biased_value = mark;
        markOop rebiased_prototype = markOopDesc::encode((JavaThread*) THREAD, mark->age(), prototype_header->bias_epoch());
        markOop res_mark = (markOop) Atomic::cmpxchg_ptr(rebiased_prototype, obj->mark_addr(), mark);
        if (res_mark == biased_value) return BIAS_REVOKED_AND_REBIASED;
      } else {
        markOop biased_value = mark;
        markOop unbiased_prototype = markOopDesc::prototype()->set_age(mark->age());
        markOop res_mark = (markOop) Atomic::cmpxchg_ptr(unbiased_prototype, obj->mark_addr(), mark);
        if (res_mark == biased_value) return BIAS_REVOKED;
      }
    }
  }
  HeuristicsResult heuristics = update_heuristics(obj(), attempt_rebias);
  if (heuristics == HR_NOT_BIASED) {
  } else if (heuristics == HR_SINGLE_REVOKE) {
    //撤销单个线程
    Klass *k = obj->klass();
    markOop prototype_header = k->prototype_header();
    if (mark->biased_locker() == THREAD && prototype_header->bias_epoch() == mark->bias_epoch()) {
      //撤销偏向当前线程的锁，调用Object.hashcode()时会走到这一步
      BiasedLocking::Condition cond = revoke_bias(obj(), false, false, (JavaThread*) THREAD, NULL);
      ((JavaThread*) THREAD)->set_cached_monitor_info(NULL);
    } else {
      //将操作push到VMOperationQueue中，自身阻塞等待jvm完成操作
      EventBiasedLockRevocation event;
      VM_RevokeBias revoke(&obj, (JavaThread*) THREAD);
      VMThread::execute(&revoke);
      return revoke.status_code(); //BiasedLocking::BIAS_REVOKED (2)
    }
  }
  //批量撤销
  EventBiasedLockClassRevocation event;
  VM_BulkRevokeBias bulk_revoke(&obj, (JavaThread*) THREAD, (heuristics == HR_BULK_REBIAS), attempt_rebias);
  VMThread::execute(&bulk_revoke);
  return bulk_revoke.status_code();
}
```

```
void VM_RevokeBias::doit() {
  if (_obj != NULL) {
    JavaThread* biased_locker = NULL;
    _status_code = revoke_bias((*_obj)(), false, false, _requesting_thread, &biased_locker);
    if (biased_locker != NULL) _biased_locker_id = JFR_THREAD_ID(biased_locker);
    return;
  } else {
    BiasedLocking::revoke_at_safepoint(_objs);
  }
}
```

```
BiasedLocking::Condition revoke_bias(oop obj, bool allow_rebias, bool is_bulk, JavaThread* requesting_thread, JavaThread** biased_locker) {
  markOop mark = obj->mark();
  if (!mark->has_bias_pattern()) return BiasedLocking::NOT_BIASED;
  uint age = mark->age();
  //构造2个mark word，一个匿名偏向模式101，一个无锁模式001
  markOop biased_prototype = markOopDesc::biased_locking_prototype()->set_age(age);
  markOop unbiased_prototype = markOopDesc::prototype()->set_age(age);
  JavaThread* biased_thread = mark->biased_locker();
  if (biased_thread == NULL) {
    if (!allow_rebias) obj->set_mark(unbiased_prototype); //此时allow_rebias为false
    return BiasedLocking::BIAS_REVOKED;
  }
  //1. 遍历jvm所有线程，看看biased_thread是死是活（JVM维护了一个集合，存放所有存放的线程）
  bool thread_is_alive = false;
  if (requesting_thread == biased_thread) { thread_is_alive = true; }
  else {
    for (JavaThread* cur_thread = Threads::first(); cur_thread != NULL; cur_thread = cur_thread->next()) {
      if (cur_thread == biased_thread) {
        thread_is_alive = true;
        break;
      }
    }
  }
  //2. 已消亡，撤销偏向锁
  if (!thread_is_alive) {
    if (allow_rebias) obj->set_mark(biased_prototype); //设置为匿名偏向
    else obj->set_mark(unbiased_prototype); //设置为无锁（此时走这个逻辑）
    return BiasedLocking::BIAS_REVOKED;
  }
  //3. 还活着
  GrowableArray<MonitorInfo*>* cached_monitor_info = get_or_compute_monitor_info(biased_thread);
  BasicLock* highest_lock = NULL;
  //遍历biased_thread线程栈中所有的Lock Record
  for (int i = 0; i < cached_monitor_info->length(); i++) {
    MonitorInfo* mon_info = cached_monitor_info->at(i);
    if (mon_info->owner() == obj) {
      //biased_thread还在执行同步代码块，需升级为轻量锁
      markOop mark = markOopDesc::encode((BasicLock*) NULL);
      highest_lock = mon_info->lock();
      highest_lock->set_displaced_header(mark); //将Lock Record中的displaced mark word置空
    } else {}
  }
  //3.1 biased_thread还在执行同步代码块，将obj设置为轻量锁加锁状态，完成锁升级
  if (highest_lock != NULL) {
    highest_lock->set_displaced_header(unbiased_prototype); //修改baised_thread栈中第1条Lock Record，displaced mark word置为无锁状态（相当于biased_thread第一次获取轻量锁时，obj的mark word）
    obj->release_set_mark(markOopDesc::encode(highest_lock)); //将修改后的Lock Record设置到obj mark word中，此时biased_lock持有的锁从偏向锁升级到轻量锁
  } else {
    //3.2 biased_thread已执行完同步代码块，将obj设置为无锁状态，完成锁撤销
    if (allow_rebias) obj->set_mark(biased_prototype); //设置为匿名偏向
    else obj->set_mark(unbiased_prototype); //设置为无锁（此时走这个逻辑）
  }

  if (biased_locker != NULL) *biased_locker = biased_thread;
  return BiasedLocking::BIAS_REVOKED;
}
```